WeChat 音视频会议系统 - 项目类图（完整重构版）

> **文档版本：** v2.0 **最后更新：** 2026-07-29 **重构说明：** 基于源码头文件全面重构，完整标注每个类的属性（名称、类型、可见性）和方法（名称、参数、返回值、可见性），使用标准 UML 符号标识所有类间关系。

一、整体架构分层

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                        UI 层（Qt Widgets 用户界面）                 │
│  LoginDialog(QDialog)  WeChatDialog(QDialog)  RoomDialog(QDialog)          │
│  UserShow(QWidget)     MyOpenGLWidget(QOpenGLWidget)                       │
├──────────────────────────────────────────────────────────────────────────────┤
│                      核心调度层（业务中枢 - 单例模式）                 │
│                            ckernel(QObject)                               │
│  协议映射表 PFUN m_netPackMap[100] │ 三路TCP连接 │ 模块生命周期管理            │
├──────────────┬──────────────┬──────────────┬──────────────┬──────────────────┤
│   音频层     │   视频采集层│  视频编解码层│   渲染层     │      同步层                  │
│AudioRead  │VideoRead  │H264Encoder│YuvGLWidget│   AVSyncManager(单例)        │
│AudioWrite│ScreenRead│H264Decoder│MyOpenGLW  │   SyncContext(结构体)        │
│(SDL+Opus)│(OpenCV)   │(FFmpeg)    │(OpenGL)    │   (音频主时钟策略)            │
├──────────────┴──────────────┴──────────────┴──────────────┴──────────────────┤
│                    网络传输层（接口抽象 + 中介者模式）                 │
│  INet(抽象接口) ──实现── TcpClient / TcpServer / UdpNet                    │
│  INetMediator(抽象中介者) ──继承── TcpClientMed / TcpServerMed / UdpMed    │
├──────────────────────────────────────────────────────────────────────────────┤
│                          工具/协议层                                  │
│  ThreadWorker(QObject)  SendVideoWorker  MD5  packdef.h(协议结构体)        │
└──────────────────────────────────────────────────────────────────────────────┘
```

UML 关系符号说明

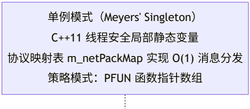
符号	关系类型	说明
`<	--`	继承（泛化）
`<	..`	实现（接口）
`*--`	组合（Composition）	强拥有关系，部分随整体创建/销毁，实线实心菱形
`o--`	聚合（Aggregation）	弱拥有关系，部分可独立于整体存在，实线空心菱形
`-->`	关联（Association）	类之间有结构关系，实线箭头
`..>`	依赖（Dependency）	类之间有使用关系，虚线箭头

可见性符号说明

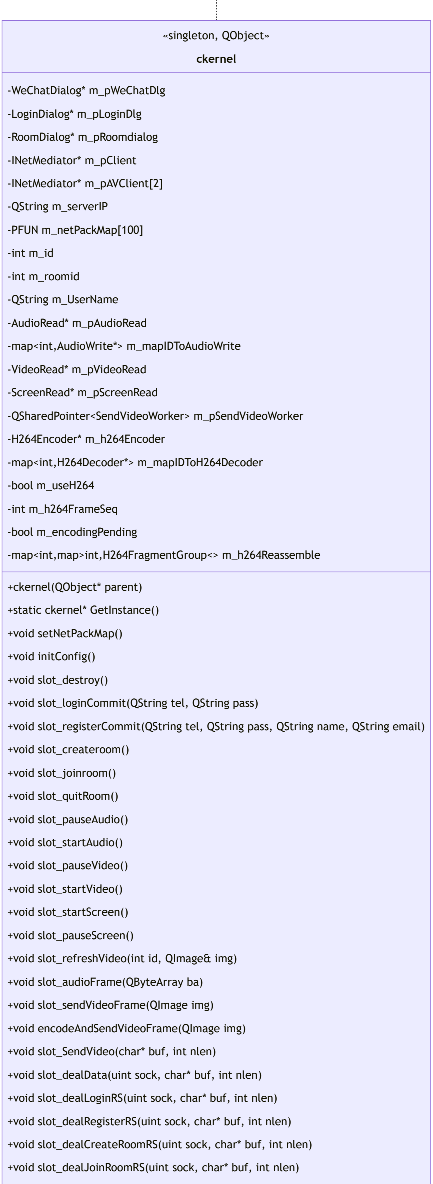
符号	可见性	说明
`+`	public	公有访问
`-`	private	私有访问
`#`	protected	受保护访问

二、完整类图（Mermaid 格式）

2.1 核心调度层



单例模式（Meyers' Singleton）
C++11 线程安全局部静态变量
协议映射表 m_netPackMap 实现 O(1) 消息分发
策略模式：PFUN 函数指针数组

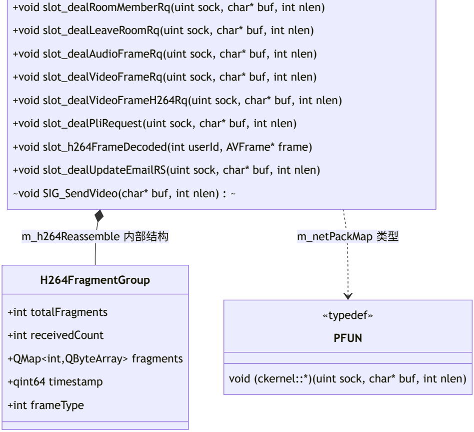


«singleton, QObject»

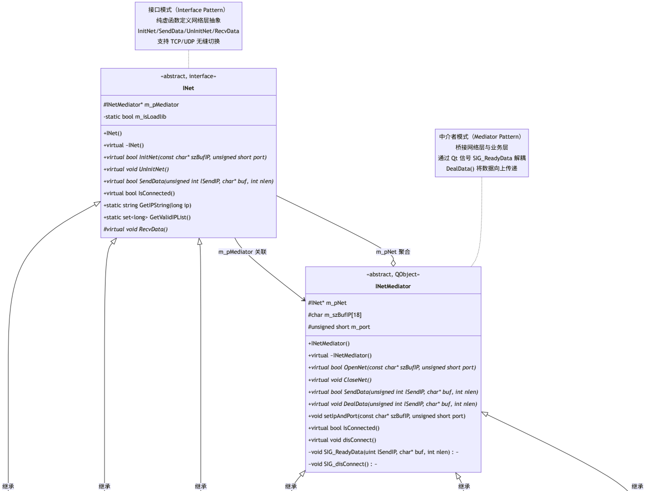
ckernel

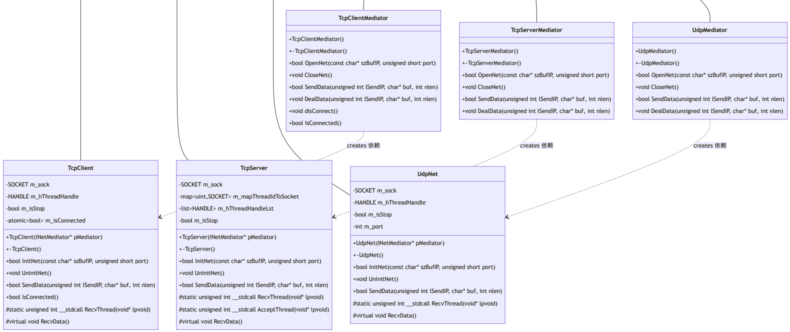
-WeChatDialog* m_pWeChatDlg
-LoginDialog* m_pLoginDlg
-RoomDialog* m_pRoomdialog
-INetMediator* m_pClient
-INetMediator* m_pAVClient[2]
-QString m_serverIP
-PFUN m_netPackMap[100]
-int m_id
-int m_roomid
-QString m_UserName
-AudioRead* m_pAudioRead
-map<int,AudioWrite*> m_mapIDToAudioWrite
-VideoRead* m_pVideoRead
-ScreenRead* m_pScreenRead
-QSharedPointer<SendVideoWorker> m_pSendVideoWorker
-H264Encoder* m_h264Encoder
-map<int,H264Decoder*> m_mapIDToH264Decoder
-bool m_useH264
-int m_h264FrameSeq
-bool m_encodingPending
-map<int,map>int,H264FragmentGroup<> m_h264Reassemble

+ckernel(QObject* parent)
+static ckernel* GetInstance()
+void setNetPackMap()
+void initConfig()
+void slot_destroy()
+void slot_loginCommit(QString tel, QString pass)
+void slot_registerCommit(QString tel, QString pass, QString name, QString email)
+void slot_createroom()
+void slot_joinroom()
+void slot_quitRoom()
+void slot_pauseAudio()
+void slot_startAudio()
+void slot_pauseVideo()
+void slot_startVideo()
+void slot_startScreen()
+void slot_pauseScreen()
+void slot_refreshVideo(int id, QImage& img)
+void slot_audioFrame(QByteArray ba)
+void slot_sendVideoFrame(QImage img)
+void encodeAndSendVideoFrame(QImage img)
+void slot_SendVideo(char* buf, int nlen)
+void slot_dealData(uint sock, char* buf, int nlen)
+void slot_dealLoginRS(uint sock, char* buf, int nlen)
+void slot_dealRegisterRS(uint sock, char* buf, int nlen)
+void slot_dealCreateRoomRS(uint sock, char* buf, int nlen)
+void slot_dealJoinRoomRS(uint sock, char* buf, int nlen)

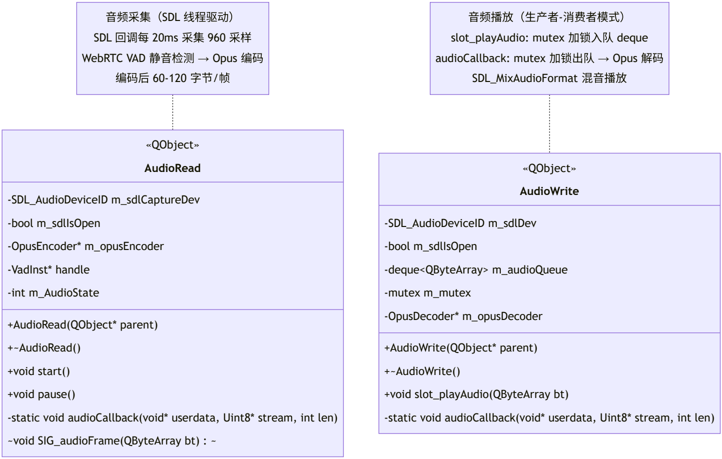

+void slot_dealRoomMemberRq(uint sock, char* buf, int nlen)
+void slot_dealLeaveRoomRq(uint sock, char* buf, int nlen)
+void slot_dealAudioFrameRq(uint sock, char* buf, int nlen)
+void slot_dealVideoFrameRq(uint sock, char* buf, int nlen)
+void slot_dealVideoFrameH264Rq(uint sock, char* buf, int nlen)
+void slot_dealPliRequest(uint sock, char* buf, int nlen)
+void slot_h264FrameDecoded(int userId, AVFrame* frame)
+void slot_dealUpdateEmailRS(uint sock, char* buf, int nlen)
~void SIG_SendVideo(char* buf, int nlen) : ~
m_h264Reassemble 内部结构
m_netPackMap 类型
H264FragmentGroup
+int totalFragments
+int receivedCount
+QMap<int,QByteArray> fragments
+qint64 timestamp
+int frameType
«typedef»
PFUN
void (ckernel::*)(uint sock, char* buf, int nlen)

2.2 网络传输层（接口抽象 + 中介者模式）

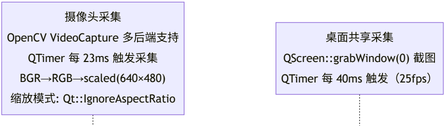

接口模式（Interface Pattern）
纯虚函数定义网络层抽象
InitNet/SendData/UnInitNet/RecvData
支持 TCP/UDP 无缝切换
«abstract, interface»
INet
#INetMediator* m_pMediator
-static bool m_isLoadlib
+INet()
+virtual -INet()
+virtual bool InitNet(const char* szBufIP, unsigned short port)
+virtual void UnInitNet()
+virtual bool SendData(unsigned int lSendIP, char* buf, int nlen)
+virtual bool IsConnected()
+static string GetIPString(long ip)
+static set<long> GetValidIPList()
#virtual void RecvData()
中介者模式（Mediator Pattern）
桥接网络层与业务层
通过 Qt 信号 SIG_ReadyData 解耦
DealData() 将数据向上传递
m_pMediator 关联
m_pNet 聚合
«abstract, QObject»
INetMediator
#INet* m_pNet
#char m_szBufIP[18]
#unsigned short m_port
+INetMediator()
+virtual -INetMediator()
+virtual bool OpenNet(const char* szBufIP, unsigned short port)
+virtual void CloseNet()
+virtual bool SendData(unsigned int lSendIP, char* buf, int nlen)
+virtual void DealData(unsigned int lSendIP, char* buf, int nlen)
+void setIpAndPort(const char* szBufIP, unsigned short port)
+virtual bool IsConnected()
+virtual void disConnect()
-void SIG_ReadyData(uint lSendIP, char* buf, int nlen) : -
-void SIG_disConnect() : -
继承
继承
继承
继承
继承
继承

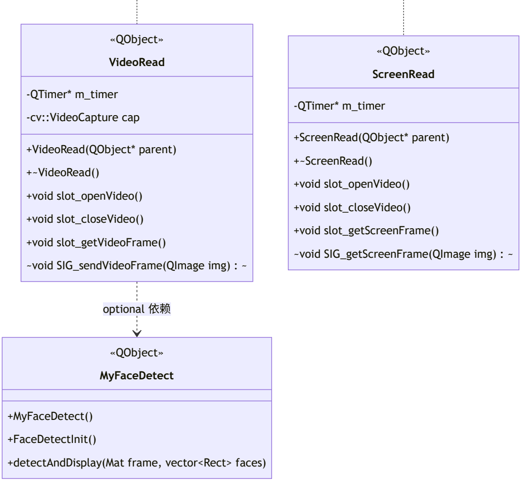

TcpClientMediator
+TcpClientMediator()
+-TcpClientMediator()
+bool OpenNet(const char* szBufIP, unsigned short port)
+void CloseNet()
+bool SendData(unsigned int lSendIP, char* buf, int nlen)
+void DealData(unsigned int lSendIP, char* buf, int nlen)
+void disConnect()
+bool IsConnected()
TcpServerMediator
+TcpServerMediator()
+-TcpServerMediator()
+bool OpenNet(const char* szBufIP, unsigned short port)
+void CloseNet()
+bool SendData(unsigned int lSendIP, char* buf, int nlen)
+void DealData(unsigned int lSendIP, char* buf, int nlen)
UdpMediator
+UdpMediator()
+-UdpMediator()
+bool OpenNet(const char* szBufIP, unsigned short port)
+void CloseNet()
+bool SendData(unsigned int lSendIP, char* buf, int nlen)
+void DealData(unsigned int lSendIP, char* buf, int nlen)
creates 依赖
creates 依赖
creates 依赖
TcpClient
-SOCKET m_sock
-HANDLE m_hThreadHandle
-bool m_isStop
-atomic<bool> m_isConnected
+TcpClient(INetMediator* pMediator)
+-TcpClient()
+bool InitNet(const char* szBufIP, unsigned short port)
+void UnInitNet()
+bool SendData(unsigned int lSendIP, char* buf, int nlen)
+bool IsConnected()
#static unsigned int __stdcall RecvThread(void* lpvoid)
#virtual void RecvData()
TcpServer
-SOCKET m_sock
-map<uint,SOCKET> m_mapThreadIdToSocket
-list<HANDLE> m_hThreadHandleLst
-bool m_isStop
+TcpServer(INetMediator* pMediator)
+-TcpServer()
+bool InitNet(const char* szBufIP, unsigned short port)
+void UnInitNet()
+bool SendData(unsigned int lSendIP, char* buf, int nlen)
#static unsigned int __stdcall RecvThread(void* lpvoid)
#static unsigned int __stdcall AcceptThread(void* lpvoid)
#virtual void RecvData()
UdpNet
-SOCKET m_sock
-HANDLE m_hThreadHandle
-bool m_isStop
-int m_port
+UdpNet(INetMediator* pMediator)
+-UdpNet()
+bool InitNet(const char* szBufIP, unsigned short port)
+void UnInitNet()
+bool SendData(unsigned int lSendIP, char* buf, int nlen)
#static unsigned int __stdcall RecvThread(void* lpvoid)
#virtual void RecvData()

2.3 音频层（SDL2 + Opus）

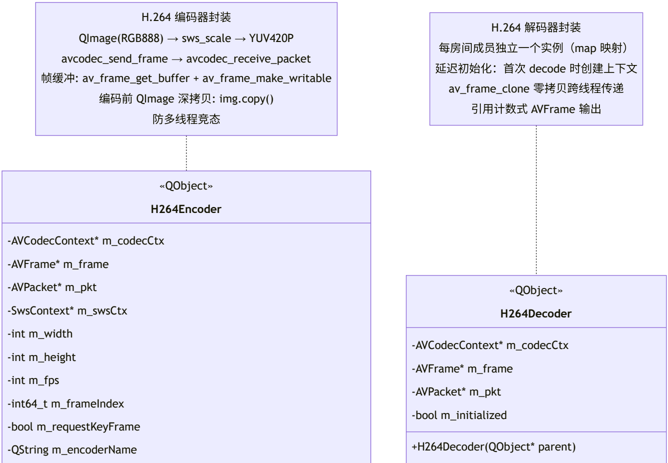

音频采集（SDL 线程驱动）
SDL 回调每 20ms 采集 960 采样
WebRTC VAD 静音检测 → Opus 编码
编码后 60-120 字节/帧
音频播放（生产者-消费者模式）
slot_playAudio: mutex 加锁入队 deque
audioCallback: mutex 加锁出队 → Opus 解码
SDL_MixAudioFormat 混音播放
«QObject»
AudioRead
-SDL_AudioDeviceID m_sdlCaptureDev
-bool m_sdlIsOpen
-OpusEncoder* m_opusEncoder
-VadInst* handle
-int m_AudioState
+AudioRead(QObject* parent)
+~AudioRead()
+void start()
+void pause()
-static void audioCallback(void* userdata, Uint8* stream, int len)
~void SIG_audioFrame(QByteArray bt) : ~
«QObject»
AudioWrite
-SDL_AudioDeviceID m_sdlDev
-bool m_sdlIsOpen
-deque<QByteArray> m_audioQueue
-mutex m_mutex
-OpusDecoder* m_opusDecoder
+AudioWrite(QObject* parent)
+~AudioWrite()
+void slot_playAudio(QByteArray bt)
-static void audioCallback(void* userdata, Uint8* stream, int len)

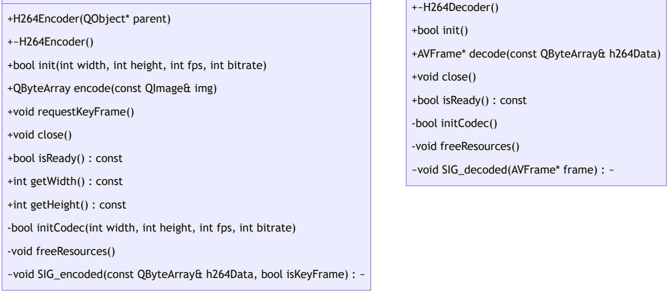
2.4 视频采集层（OpenCV + Qt Screen）


摄像头采集
OpenCV VideoCapture 多后端支持
QTimer 每 23ms 触发采集
BGR→RGB→scaled(640×480)
缩放模式: Qt::IgnoreAspectRatio
桌面共享采集
QScreen::grabWindow(0) 截图
QTimer 每 40ms 触发（25fps）


«QObject»
VideoRead
-QTimer* m_timer
-cv::VideoCapture cap
+VideoRead(QObject* parent)
+~VideoRead()
+void slot_openVideo()
+void slot_closeVideo()
+void slot_getVideoFrame()
~void SIG_sendVideoFrame(QImage img) : ~
«QObject»
ScreenRead
-QTimer* m_timer
+ScreenRead(QObject* parent)
+~ScreenRead()
+void slot_openVideo()
+void slot_closeVideo()
+void slot_getScreenFrame()
~void SIG_getScreenFrame(QImage img) : ~
optional 依赖
«QObject»
MyFaceDetect
+MyFaceDetect()
+FaceDetectInit()
+detectAndDisplay(Mat frame, vector<Rect> faces)

2.5 视频编解码层（FFmpeg H.264）

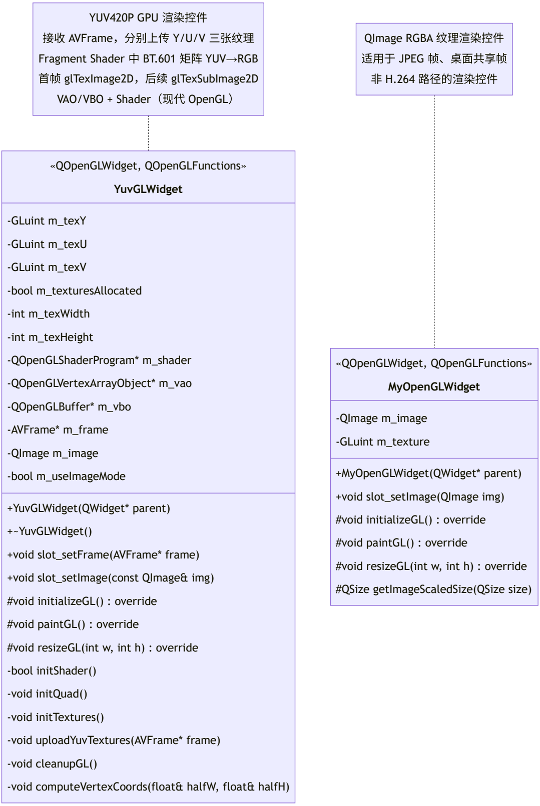

H.264 编码器封装
QImage(RGB888) → sws_scale → YUV420P
avcodec_send_frame → avcodec_receive_packet
帧缓冲: av_frame_get_buffer + av_frame_make_writable
编码前 QImage 深拷贝: img.copy()
防多线程竞态
H.264 解码器封装
每房间成员独立一个实例（map 映射）
延迟初始化：首次 decode 时创建上下文
av_frame_clone 零拷贝跨线程传递
引用计数式 AVFrame 输出
«QObject»
H264Encoder
-AVCodecContext* m_codecCtx
-AVFrame* m_frame
-AVPacket* m_pkt
-SwsContext* m_swsCtx
-int m_width
-int m_height
-int m_fps
-int64_t m_frameIndex
-bool m_requestKeyFrame
-QString m_encoderName
«QObject»
H264Decoder
-AVCodecContext* m_codecCtx
-AVFrame* m_frame
-AVPacket* m_pkt
-bool m_initialized
+H264Decoder(QObject* parent)



```
+H264Encoder(QObject* parent)
+~H264Encoder()
+bool init(int width, int height, int fps, int bitrate)
+QByteArray encode(const QImage& img)
+void requestKeyFrame()
+void close()
+bool isReady() : const
+int getWidth() : const
+int getHeight() : const
-bool initCodec(int width, int height, int fps, int bitrate)
-void freeResources()
~void SIG_encoded(const QByteArray& h264Data, bool isKeyFrame) : ~
```

```
+~H264Decoder()
+bool init()
+AVFrame* decode(const QByteArray& h264Data)
+void close()
+bool isReady() : const
-bool initCodec()
-void freeResources()
~void SIG_decoded(AVFrame* frame) : ~
```

2.6 渲染层（OpenGL）



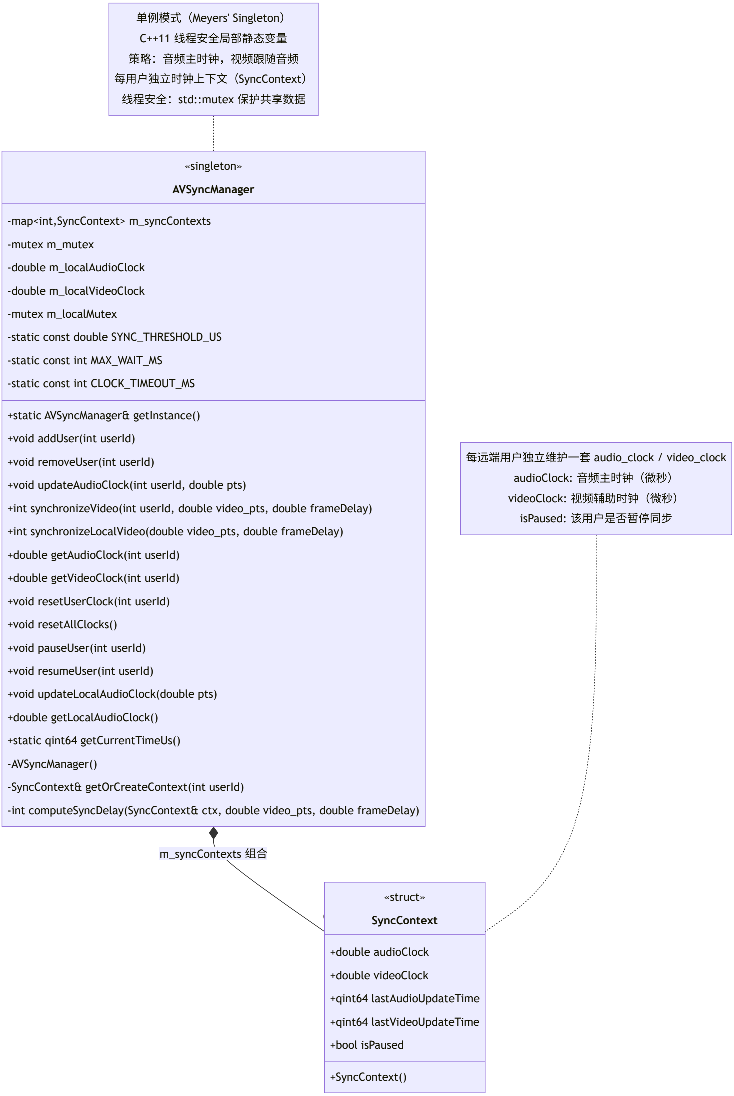
YUV420P GPU 渲染控件
接收 AVFrame，分别上传 Y/U/V 三张纹理
Fragment Shader 中 BT.601 矩阵 YUV→RGB
首帧 glTexImage2D，后续 glTexSubImage2D
VAO/VBO + Shader（现代 OpenGL）

```
«QOpenGLWidget, QOpenGLFunctions»
YuvGLWidget
-GLuint m_texY
-GLuint m_texU
-GLuint m_texV
-bool m_texturesAllocated
-int m_texWidth
-int m_texHeight
-QOpenGLShaderProgram* m_shader
-QOpenGLVertexArrayObject* m_vao
-QOpenGLBuffer* m_vbo
-AVFrame* m_frame
-QImage m_image
-bool m_useImageMode
+YuvGLWidget(QWidget* parent)
+~YuvGLWidget()
+void slot_setFrame(AVFrame* frame)
+void slot_setImage(const QImage& img)
#void initializeGL() : override
#void paintGL() : override
#void resizeGL(int w, int h) : override
-bool initShader()
-void initQuad()
-void initTextures()
-void uploadYuvTextures(AVFrame* frame)
-void cleanupGL()
-void computeVertexCoords(float& halfW, float& halfH)
```

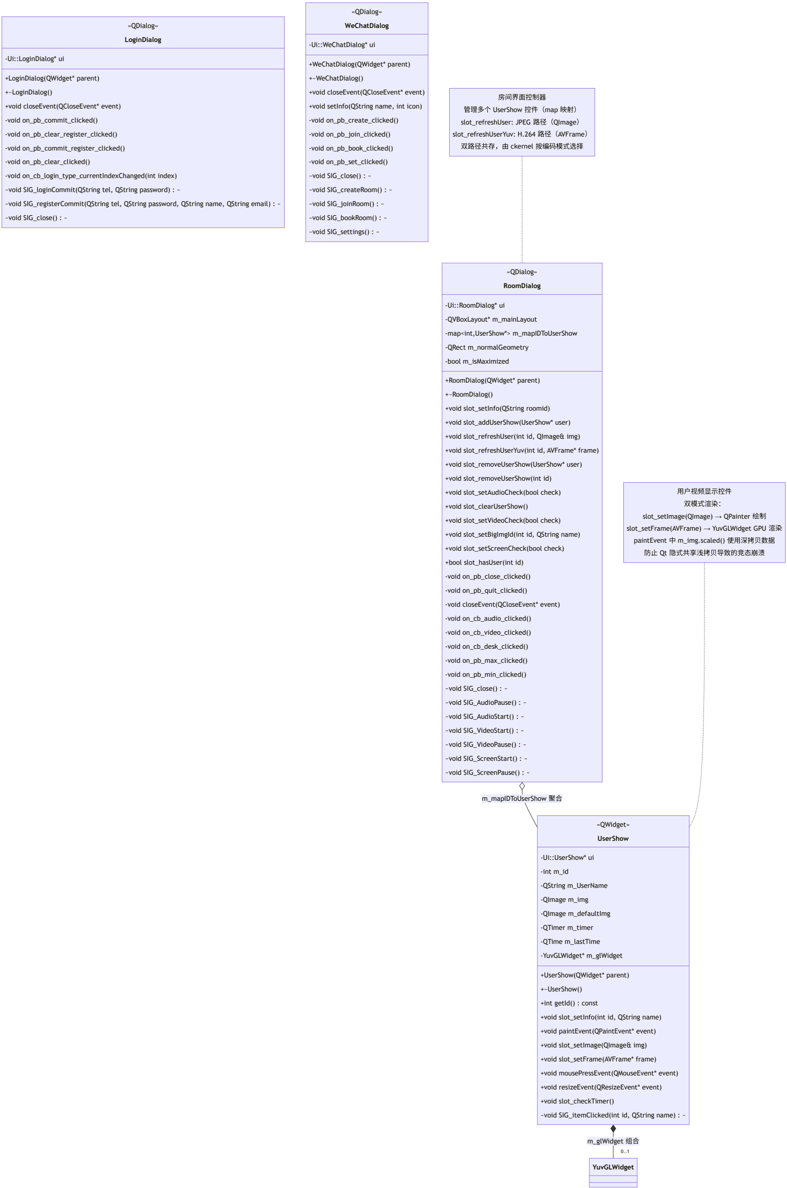
QImage RGBA 纹理渲染控件
适用于 JPEG 帧、桌面共享帧
非 H.264 路径的渲染控件

```
«QOpenGLWidget, QOpenGLFunctions»
MyOpenGLWidget
-QImage m_image
-GLuint m_texture
+MyOpenGLWidget(QWidget* parent)
+void slot_setImage(QImage img)
#void initializeGL() : override
#void paintGL() : override
#void resizeGL(int w, int h) : override
#QSize getImageScaledSize(QSize size)
```

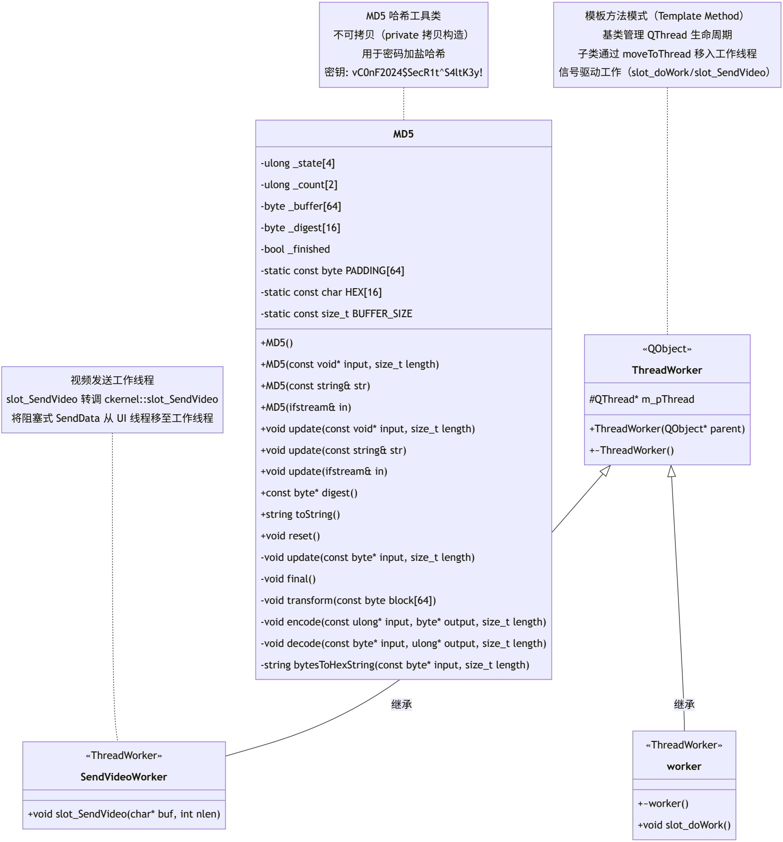
2.7 音视频同步层（单例模式）


单例模式（Meyers' Singleton）
C++11 线程安全局部静态变量
策略：音频主时钟，视频跟随音频
每用户独立时钟上下文（SyncContext）
线程安全：std::mutex 保护共享数据
«singleton»
AVSyncManager
-map<int,SyncContext> m_syncContexts
-mutex m_mutex
-double m_localAudioClock
-double m_localVideoClock
-mutex m_localMutex
-static const double SYNC_THRESHOLD_US
-static const int MAX_WAIT_MS
-static const int CLOCK_TIMEOUT_MS
+static AVSyncManager& getInstance()
+void addUser(int userId)
+void removeUser(int userId)
+void updateAudioClock(int userId, double pts)
+int synchronizeVideo(int userId, double video_pts, double frameDelay)
+int synchronizeLocalVideo(double video_pts, double frameDelay)
+double getAudioClock(int userId)
+double getVideoClock(int userId)
+void resetUserClock(int userId)
+void resetAllClocks()
+void pauseUser(int userId)
+void resumeUser(int userId)
+void updateLocalAudioClock(double pts)
+double getLocalAudioClock()
+static qint64 getCurrentTimeUs()
-AVSyncManager()
-SyncContext& getOrCreateContext(int userId)
-int computeSyncDelay(SyncContext& ctx, double video_pts, double frameDelay)
每远端用户独立维护一套 audio_clock / video_clock
audioClock: 音频主时钟（微秒）
videoClock: 视频辅助时钟（微秒）
isPaused: 该用户是否暂停同步
m_syncContexts 组合
«struct»
SyncContext
+double audioClock
+double videoClock
+qint64 lastAudioUpdateTime
+qint64 lastVideoUpdateTime
+bool isPaused
+SyncContext()

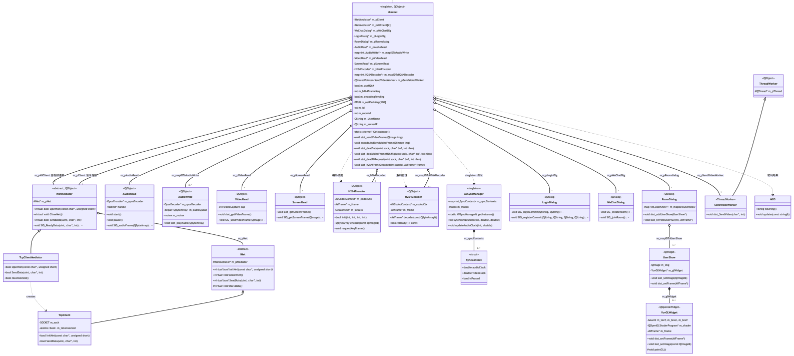
2.8 UI 层


«QDialog»
LoginDialog
-Ui::LoginDialog* ui
+LoginDialog(QWidget* parent)
+~LoginDialog()
+void closeEvent(QCloseEvent* event)
-void on_pb_commit_clicked()
-void on_pb_clear_register_clicked()
-void on_pb_commit_register_clicked()
-void on_pb_clear_clicked()
-void on_cb_login_type_currentIndexChanged(int index)
~void SIG_loginCommit(QString tel, QString password) : ~
~void SIG_registerCommit(QString tel, QString password, QString name, QString email) : ~
~void SIG_close() : ~
«QDialog»
WeChatDialog
-Ui::WeChatDialog* ui
+WeChatDialog(QWidget* parent)
+~WeChatDialog()
+void closeEvent(QCloseEvent* event)
+void setInfo(QString name, int icon)
-void on_pb_create_clicked()
-void on_pb_join_clicked()
-void on_pb_book_clicked()
-void on_pb_set_clicked()
~void SIG_close() : ~
~void SIG_createRoom() : ~
~void SIG_joinRoom() : ~
~void SIG_bookRoom() : ~
~void SIG_settings() : ~
房间界面控制器
管理多个 UserShow 控件（map 映射）
slot_refreshUser: JPEG 路径（QImage）
slot_refreshUserYuv: H.264 路径（AVFrame）
双路径共存，由 ckernel 按编码模式选择
«QDialog»
RoomDialog
-Ui::RoomDialog* ui
-QVBoxLayout* m_mainLayout
-map<int,UserShow*> m_mapIDToUserShow
-QRect m_normalGeometry
-bool m_isMaximized
+RoomDialog(QWidget* parent)
+~RoomDialog()
+void slot_setInfo(QString roomid)
+void slot_addUserShow(UserShow* user)
+void slot_refreshUser(int id, QImage& img)
+void slot_refreshUserYuv(int id, AVFrame* frame)
+void slot_removeUserShow(UserShow* user)
+void slot_removeUserShow(int id)
+void slot_setAudioCheck(bool check)
+void slot_clearUserShow()
+void slot_setVideoCheck(bool check)
+void slot_setBigImgId(int id, QString name)
+void slot_setScreenCheck(bool check)
+bool slot_hasUser(int id)
-void on_pb_close_clicked()
-void on_pb_quit_clicked()
-void closeEvent(QCloseEvent* event)
-void on_cb_audio_clicked()
-void on_cb_video_clicked()
-void on_cb_desk_clicked()
-void on_pb_max_clicked()
-void on_pb_min_clicked()
~void SIG_close() : ~
~void SIG_AudioPause() : ~
~void SIG_AudioStart() : ~
~void SIG_VideoStart() : ~
~void SIG_VideoPause() : ~
~void SIG_ScreenStart() : ~
~void SIG_ScreenPause() : ~
用户视频显示控件
双模式渲染：
slot_setImage(QImage) → QPainter 绘制
slot_setFrame(AVFrame) → YuvGLWidget GPU 渲染
paintEvent 中 m_img.scaled() 使用深拷贝数据
防止 Qt 隐式共享浅拷贝导致的竞态崩溃
m_mapIDToUserShow 聚合
«QWidget»
UserShow
-Ui::UserShow* ui
-int m_id
-QString m_UserName
-QImage m_img
-QImage m_defaultImg
-QTimer m_timer
-QTime m_lastTime
-YuvGLWidget* m_glWidget
+UserShow(QWidget* parent)
+~UserShow()
+int getId() : const
+void slot_setInfo(int id, QString name)
+void paintEvent(QPaintEvent* event)
+void slot_setImage(QImage& img)
+void slot_setFrame(AVFrame* frame)
+void mousePressEvent(QMouseEvent* event)
+void resizeEvent(QResizeEvent* event)
+void slot_checkTimer()
~void SIG_itemClicked(int id, QString name) : ~
m_glWidget 组合
0..1
YuvGLWidget

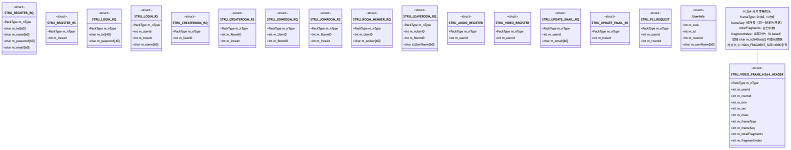
2.9 工具/线程层


MD5 哈希工具类
不可拷贝（private 拷贝构造）
用于密码加盐哈希
密钥: vC0nF2024$SecR1t^S4ltK3y!
模板方法模式（Template Method）
基类管理 QThread 生命周期
子类通过 moveToThread 移入工作线程
信号驱动工作（slot_doWork/slot_SendVideo）
MD5
-ulong _state[4]
-ulong _count[2]
-byte _buffer[64]
-byte _digest[16]
-bool _finished
-static const byte PADDING[64]
-static const char HEX[16]
-static const size_t BUFFER_SIZE
+MD5()
+MD5(const void* input, size_t length)
+MD5(const string& str)
+MD5(ifstream& in)
+void update(const void* input, size_t length)
+void update(const string& str)
+void update(ifstream& in)
+const byte* digest()
+string toString()
+void reset()
-void update(const byte* input, size_t length)
-void final()
-void transform(const byte block[64])
-void encode(const ulong* input, byte* output, size_t length)
-void decode(const byte* input, ulong* output, size_t length)
-string bytesToHexString(const byte* input, size_t length)
«QObject»
ThreadWorker
#QThread* m_pThread
+ThreadWorker(QObject* parent)
+~ThreadWorker()
视频发送工作线程
slot_SendVideo 转调 ckernel::slot_SendVideo
将阻塞式 SendData 从 UI 线程移至工作线程
继承
继承
«ThreadWorker»
SendVideoWorker
+void slot_SendVideo(char* buf, int nlen)
«ThreadWorker»
worker
+~worker()
+void slot_doWork()

三、完整类关系总图

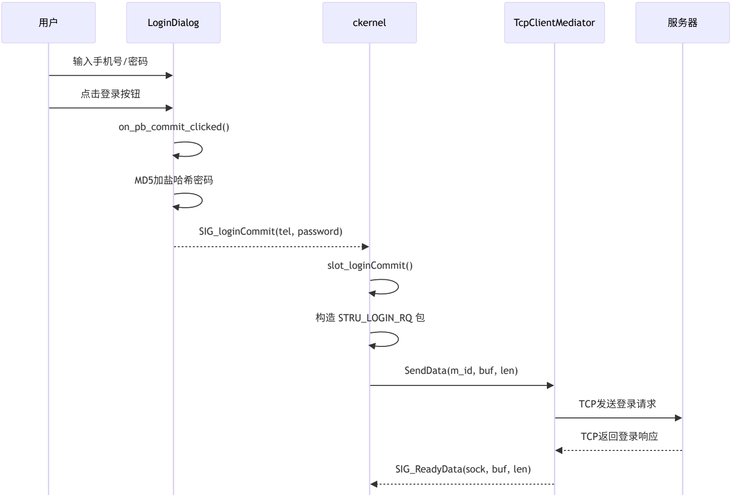

«singleton, QObject»
ckernel
-INetMediator* m_pClient
-INetMediator* m_pAVClient[2]
-WeChatDialog* m_pWeChatDlg
-LoginDialog* m_pLoginDlg
-RoomDialog* m_pRoomdialog
-AudioRead* m_pAudioRead
-map<int,AudioWrite*> m_mapIDToAudioWrite
-VideoRead* m_pVideoRead
-ScreenRead* m_pScreenRead
-H264Encoder* m_h264Encoder
-map<int,H264Decoder*> m_mapIDToH264Decoder
-QSharedPointer<SendVideoWorker> m_pSendVideoWorker
-bool m_useH264
-int m_h264FrameSeq
-bool m_encodingPending
-PFUN m_netPackMap[100]
-int m_id
-int m_roomid
-QString m_UserName
-QString m_serverIP
+static ckernel* GetInstance()
+void slot_sendVideoFrame(QImage img)
+void encodeAndSendVideoFrame(QImage img)
+void slot_dealData(uint sock, char* buf, int nlen)
+void slot_dealVideoFrameH264Rq(uint sock, char* buf, int nlen)
+void slot_dealPliRequest(uint sock, char* buf, int nlen)
+void slot_h264FrameDecoded(int userid, AVFrame* frame)
m_pAVClient 音视频连接
m_pClient 信令连接
m_pAudioRead
m_mapIDToAudioWrite
m_pVideoRead
m_pScreenRead
编码调度
m_h264Encoder
解码管理
m_mapIDToH264Decoder
singleton 访问
m_pLoginDlg
m_pWeChatDlg
m_pRoomdialog
m_pSendVideoWorker
密码哈希
«abstract, QObject»
INetMediator
#INet* m_pNet
+virtual bool OpenNet(const char*, unsigned short)
+virtual void CloseNet()
+virtual bool SendData(uint, char*, int)
-void SIG_ReadyData(uint, char*, int) : -
«QObject»
AudioRead
-OpusEncoder* m_opusEncoder
-VadInst* handle
+void start()
+void pause()
-void SIG_audioFrame(QByteArray) : -
«QObject»
AudioWrite
-OpusDecoder* m_opusDecoder
-deque<QByteArray> m_audioQueue
-mutex m_mutex
+void slot_playAudio(QByteArray)
«QObject»
VideoRead
-cv::VideoCapture cap
+void slot_getVideoFrame()
-void SIG_sendVideoFrame(QImage) : -
«QObject»
ScreenRead
+void slot_getScreenFrame()
-void SIG_getScreenFrame(QImage) : -
«QObject»
H264Encoder
-AVCodecContext* m_codecCtx
-AVFrame* m_frame
-SwsContext* m_swsCtx
+bool init(int, int, int, int)
+QByteArray encode(const QImage&)
+void requestKeyFrame()
«QObject»
H264Decoder
-AVCodecContext* m_codecCtx
-AVFrame* m_frame
+AVFrame* decode(const QByteArray&)
+bool isReady() : const
«singleton»
AVSyncManager
-map<int,SyncContext> m_syncContexts
-mutex m_mutex
+static AVSyncManager& getInstance()
+int synchronizeVideo(int, double, double)
+void updateAudioClock(int, double)
m_syncContexts
«struct»
SyncContext
+double audioClock
+double videoClock
+bool isPaused
«QDialog»
LoginDialog
-void SIG_loginCommit(QString, QString) : -
-void SIG_registerCommit(QString, QString, QString, QString) : -
«QDialog»
WeChatDialog
-void SIG_createRoom() : -
-void SIG_joinRoom() : -
«QDialog»
RoomDialog
-map<int,UserShow*> m_mapIDToUserShow
+void slot_addUserShow(UserShow*)
+void slot_refreshUserYuv(int, AVFrame*)
m_mapIDToUserShow
«QWidget»
UserShow
-QImage m_img
-YuvGLWidget* m_glWidget
+void slot_setImage(QImage&)
+void slot_setFrame(AVFrame*)
m_glWidget
«QOpenGLWidget»
YuvGLWidget
-GLuint m_texY, m_texU, m_texV
-QOpenGLShaderProgram* m_shader
-AVFrame* m_frame
+void slot_setFrame(AVFrame*)
+void slot_setImage(const QImage&)
#void paintGL()
«ThreadWorker»
SendVideoWorker
+void slot_SendVideo(char*, int)
«QObject»
ThreadWorker
#QThread* m_pThread
MD5
+string toString()
+void update(const string&)
TcpClientMediator
+bool OpenNet(const char*, unsigned short)
+bool SendData(uint, char*, int)
+bool IsConnected()
creates
TcpClient
-SOCKET m_sock
-atomic<bool> m_isConnected
+bool InitNet(const char*, unsigned short)
+bool SendData(uint, char*, int)
m_pNet
«abstract»
INet
#INetMediator* m_pMediator
+virtual bool InitNet(const char*, unsigned short)
+virtual void UnInitNet()
+virtual bool SendData(uint, char*, int)
#virtual void RecvData()

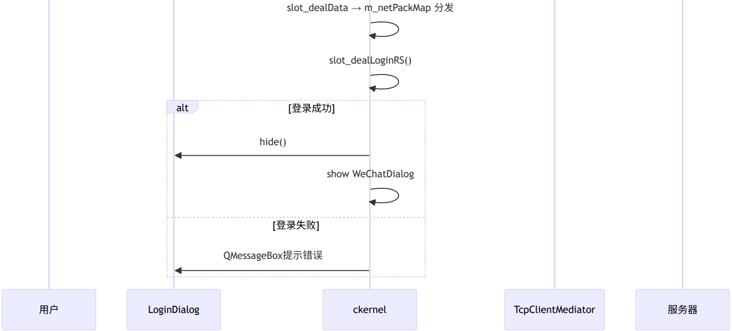
四、协议结构体定义


«struct»
STRU_REGISTER_RQ
+PackType m_nType
+char m_tel[60]
+char m_name[60]
+char m_password[60]
+char m_email[60]
«struct»
STRU_REGISTER_RS
+PackType m_nType
+int m_lresult
«struct»
STRU_LOGIN_RQ
+PackType m_nType
+char m_tel[40]
+char m_password[40]
«struct»
STRU_LOGIN_RS
+PackType m_nType
+int m_userid
+int m_lresult
+char m_name[60]
«struct»
STRU_CREATEROOM_RQ
+PackType m_nType
+int m_UserID
«struct»
STRU_CREATEROOM_RS
+PackType m_nType
+int m_RoomID
+int m_lresult
«struct»
STRU_JOINROOM_RQ
+PackType m_nType
+int m_UserID
+int m_RoomID
«struct»
STRU_JOINROOM_RS
+PackType m_nType
+int m_RoomID
+int m_lresult
«struct»
STRU_ROOM_MEMBER_RQ
+PackType m_nType
+int m_UserID
+char m_szUser[60]
«struct»
STRU_LEAVEROOM_RQ
+PackType m_nType
+int m_nUserID
+int m_RoomID
+char szUserName[60]
«struct»
STRU_AUDIO_REGISTER
+PackType m_nType
+int m_userid
«struct»
STRU_VIDEO_REGISTER
+PackType m_nType
+int m_userid
«struct»
STRU_UPDATE_EMAIL_RQ
+PackType m_nType
+int m_userid
+char m_email[60]
«struct»
STRU_UPDATE_EMAIL_RS
+PackType m_nType
+int m_lresult
«struct»
STRU_PLI_REQUEST
+PackType m_nType
+int m_userid
+int m_roomid
«struct»
UserInfo
+int m_sock
+int m_id
+int m_roomid
+char m_userName[60]
H.264 分片传输包头
frameType: 0=I帧, 1=P帧
frameSeq: 帧序号（同一帧多片共享）
totalFragments: 总分片数
fragmentIndex: 当前分片（0-based）
后接 char m_h264Data[] 可变长数据
分片大小: H264_FRAGMENT_SIZE=4096字节
«struct»
STRU_VIDEO_FRAME_H264_HEADER
+PackType m_nType
+int m_userid
+int m_roomid
+int m_min
+int m_sec
+int m_msec
+int m_frameType
+int m_frameSeq
+int m_totalFragments
+int m_fragmentIndex

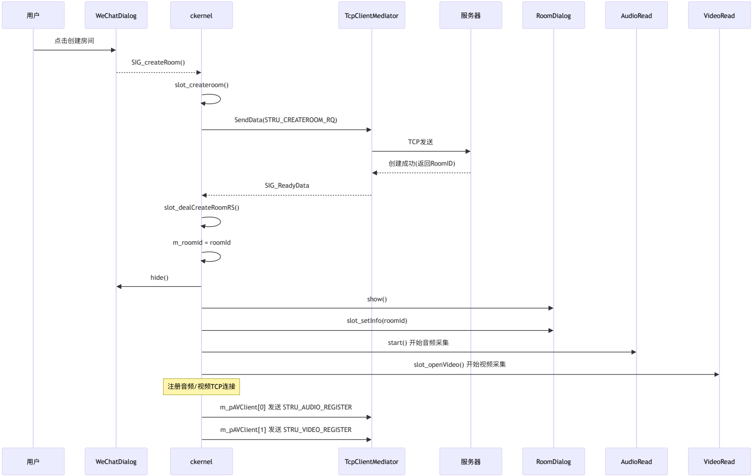
五、关键业务流程 - 信号槽交互路径

5.1 用户登录流程

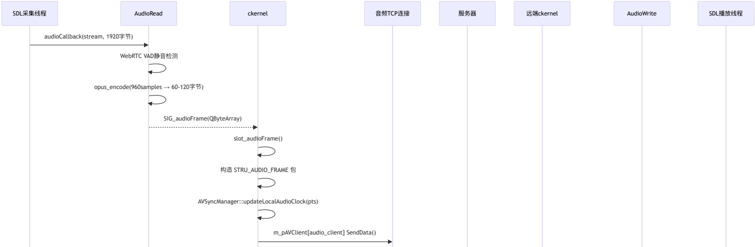

用户
LoginDialog
ckernel
TcpClientMediator
服务器
输入手机号/密码
点击登录按钮
on_pb_commit_clicked()
MD5加盐哈希密码
SIG_loginCommit(tel, password)
slot_loginCommit()
构造 STRU_LOGIN_RQ 包
SendData(m_id, buf, len)
TCP发送登录请求
TCP返回登录响应
SIG_ReadyData(sock, buf, len)

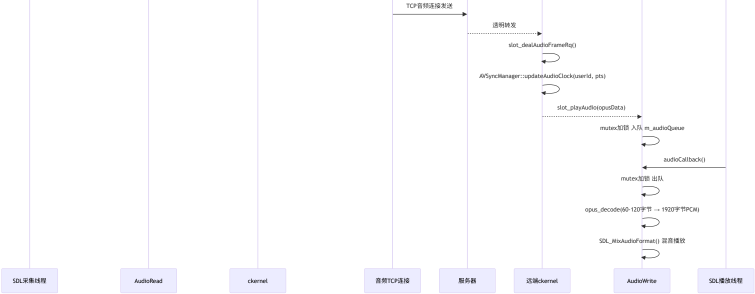

slot_dealData → m_netPackMap 分发
slot_dealLoginRS()
alt
[登录成功]
hide()
show WeChatDialog
[登录失败]
QMessageBox提示错误
用户
LoginDialog
ckernel
TcpClientMediator
服务器

5.2 创建/加入房间流程

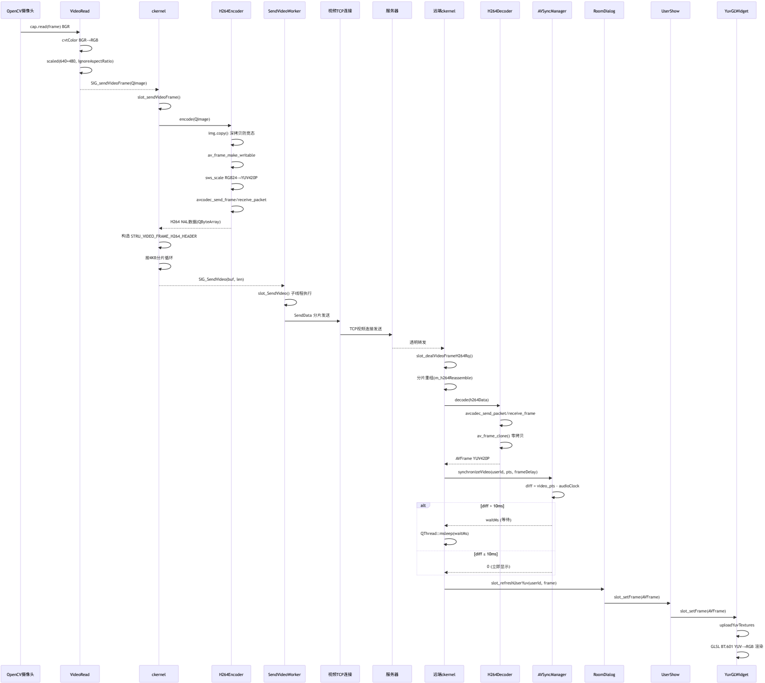

用户
WeChatDialog
ckernel
TcpClientMediator
服务器
RoomDialog
AudioRead
VideoRead
点击创建房间
SIG_createRoom()
slot_createroom()
SendData(STRU_CREATEROOM_RQ)
TCP发送
创建成功(返回RoomID)
SIG_ReadyData
slot_dealCreateRoomRS()
m_roomid = roomId
hide()
show()
slot_setInfo(roomid)
start() 开始音频采集
slot_openVideo() 开始视频采集
注册音频/视频TCP连接
m_pAVClient[0] 发送 STRU_AUDIO_REGISTER
m_pAVClient[1] 发送 STRU_VIDEO_REGISTER
用户
WeChatDialog
ckernel
TcpClientMediator
服务器
RoomDialog
AudioRead
VideoRead

5.3 音频采集到播放全链路

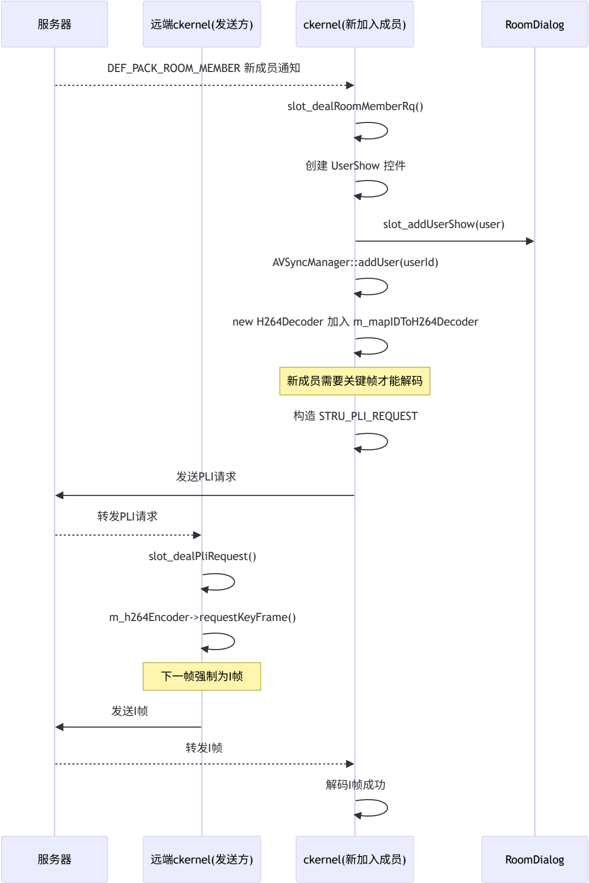

SDL采集线程
AudioRead
ckernel
音频TCP连接
服务器
远端ckernel
AudioWrite
SDL播放线程
audioCallback(stream, 1920字节)
WebRTC VAD静音检测
opus_encode(960samples → 60-120字节)
SIG_audioFrame(QByteArray)
slot_audioFrame()
构造 STRU_AUDIO_FRAME 包
AVSyncManager::updateLocalAudioClock(pts)
m_pAVClient[audio_client] SendData()


TCP音频连接发送
透明转发
slot_dealAudioFrameRq()
AVSyncManager::updateAudioClock(userId, pts)
slot_playAudio(opusData)
mutex加锁 入队 m_audioQueue
audioCallback()
mutex加锁 出队
opus_decode(60-120字节 → 1920字节PCM)
SDL_MixAudioFormat() 混音播放
SDL采集线程
AudioRead
ckernel
音频TCP连接
服务器
远端ckernel
AudioWrite
SDL播放线程

5.4 视频采集到渲染全链路


OpenCV摄像头
VideoRead
ckernel
H264Encoder
SendVideoWorker
视频TCP连接
服务器
远端ckernel
H264Decoder
AVSyncManager
RoomDialog
UserShow
YuvGLWidget
cap.read(frame) BGR
cvtColor BGR→RGB
scaled(640×480, IgnoreAspectRatio)
SIG_sendVideoFrame(QImage)
slot_sendVideoFrame()
encode(QImage)
img.copy() 深拷贝防竞态
av_frame_make_writable
sws_scale RGB24→YUV420P
avcodec_send_frame/receive_packet
H264 NAL数据(QByteArray)
构造 STRU_VIDEO_FRAME_H264_HEADER
按4KB分片循环
SIG_SendVideo(buf, len)
slot_SendVideo() 子线程执行
SendData 分片发送
TCP视频连接发送
透明转发
slot_dealVideoFrameH264Rq()
分片重组(m_h264Reassemble)
decode(h264Data)
avcodec_send_packet/receive_frame
av_frame_clone() 零拷贝
AVFrame YUV420P
synchronizeVideo(userId, pts, frameDelay)
diff = video_pts - audioClock
alt
[diff > 10ms]
waitMs (等待)
QThread::msleep(waitMs)
[diff ≤ 10ms]
0 (立即显示)
slot_refreshUserYuv(userId, frame)
slot_setFrame(AVFrame)
slot_setFrame(AVFrame)
uploadYuvTextures
GLSL BT.601 YUV→RGB 渲染
OpenCV摄像头
VideoRead
ckernel
H264Encoder
SendVideoWorker
视频TCP连接
服务器
远端ckernel
H264Decoder
AVSyncManager
RoomDialog
UserShow
YuvGLWidget

5.5 PLI 关键帧请求流程


服务器
远端ckernel(发送方)
ckernel(新加入成员)
RoomDialog
DEF_PACK_ROOM_MEMBER 新成员通知
slot_dealRoomMemberRq()
创建 UserShow 控件
slot_addUserShow(user)
AVSyncManager::addUser(userId)
new H264Decoder 加入 m_mapIDToH264Decoder
新成员需要关键帧才能解码
构造 STRU_PLI_REQUEST
发送PLI请求
转发PLI请求
slot_dealPliRequest()
m_h264Encoder->requestKeyFrame()
下一帧强制为I帧
发送I帧
转发I帧
解码I帧成功
服务器
远端ckernel(发送方)
ckernel(新加入成员)
RoomDialog

六、设计模式总结

设计模式	应用位置	具体实现	UML 关系	说明
接口模式	INet → TcpClient/TcpServer/UdpNet	纯虚函数 InitNet/SendData/UnInitNet/RecvData	`INet <	-- TcpClient`
中介者模式	INetMediator → TcpClientMediator 等	DealData() 桥接网络层与业务层	`INetMediator o-- INet`	Qt 信号 SIG_ReadyData 解耦网络与业务
单例模式	ckernel::GetInstance()	Meyers 单例（C++11 线程安全局部静态变量）	`ckernel ..> ckernel`	全局唯一业务中枢
单例模式	AVSyncManager::getInstance()	Meyers 单例（C++11 线程安全）	`AVSyncManager ..> AVSyncManager`	全局唯一同步管理器
观察者模式	Qt 信号槽机制	connect(sender, SIGNAL, receiver, SLOT)	`-->> SIG_xxx`	模块间松耦合通信
策略模式	m_netPackMap[100] 协议映射表	PFUN 函数指针数组，O(1) 消息分发	`ckernel ..> PFUN`	m_netPackMap[type - 10000] = slot_dealXxx
生产者-消费者	AudioWrite::m_audioQueue	std::deque + std::mutex	`AudioWrite *-- m_audioQueue`	SDL 回调线程消费，Qt 主线程生产
模板方法	ThreadWorker → SendVideoWorker	基类管理 QThread 生命周期	`ThreadWorker <	-- SendVideoWorker`
零拷贝	H264Decoder::decode() → AVFrame	av_frame_clone 引用计数	`H264Decoder ..> AVFrame`	跨线程传递不复制像素数据
组合模式	AVSyncManager → SyncContext	每用户独立同步上下文	`AVSyncManager *-- SyncContext`	部分随整体创建/销毁

七、协议类型常量表

常量名	值	方向	对应结构体	说明
DEF_PACK_REGISTER_RQ	10000	C→S	STRU_REGISTER_RQ	注册请求
DEF_PACK_REGISTER_RS	10001	S→C	STRU_REGISTER_RS	注册响应
DEF_PACK_LOGIN_RQ	10002	C→S	STRU_LOGIN_RQ	登录请求
DEF_PACK_LOGIN_RS	10003	S→C	STRU_LOGIN_RS	登录响应
DEF_PACK_CREATEROOM_RQ	10004	C→S	STRU_CREATEROOM_RQ	创建房间请求
DEF_PACK_CREATEROOM_RS	10005	S→C	STRU_CREATEROOM_RS	创建房间响应
DEF_PACK_JOINROOM_RQ	10006	C→S	STRU_JOINROOM_RQ	加入房间请求
DEF_PACK_JOINROOM_RS	10007	S→C	STRU_JOINROOM_RS	加入房间响应
DEF_PACK_ROOM_MEMBER	10008	S→C	STRU_ROOM_MEMBER_RQ	房间成员通知
DEF_PACK_AUDIO_FRAME	10009	C→S→C	(内联结构)	音频帧(转发)
DEF_PACK_VIDEO_FRAME	10010	C→S→C	(内联结构)	视频帧-JPEG(转发)
DEF_PACK_LEAVEROOM_RQ	10011	C→S	STRU_LEAVEROOM_RQ	离开房间请求
DEF_PACK_LEAVEROOM_RS	10012	S→C	STRU_ROOM_MEMBER_RQ	离开房间通知
DEF_PACK_AUDIO_REGISTER	10013	C→S	STRU_AUDIO_REGISTER	音频通道注册
DEF_PACK_VIDEO_REGISTER	10014	C→S	STRU_VIDEO_REGISTER	视频通道注册
DEF_PACK_UPDATE_EMAIL_RQ	10015	C→S	STRU_UPDATE_EMAIL_RQ	补全邮箱请求
DEF_PACK_UPDATE_EMAIL_RS	10016	S→C	STRU_UPDATE_EMAIL_RS	补全邮箱响应
DEF_PACK_VIDEO_FRAME_H264	10017	C→S→C	STRU_VIDEO_FRAME_H264_HEADER	视频帧-H264(转发)
DEF_PACK_PLI_REQUEST	10018	C→S→C	STRU_PLI_REQUEST	PLI关键帧请求
DEF_PACK_VIDEO_SPSPPS	10019	C→S→C	(可选)	H264 SPS/PPS参数集

八、类间关系完整说明

8.1 继承关系（泛化）

父类	子类	说明
INet	TcpClient	TCP 客户端实现
INet	TcpServer	TCP 服务端实现
INet	UdpNet	UDP 网络实现
INetMediator	TcpClientMediator	TCP 客户端中介者
INetMediator	TcpServerMediator	TCP 服务端中介者
INetMediator	UdpMediator	UDP 中介者
ThreadWorker	SendVideoWorker	视频发送工作线程
ThreadWorker	worker	示例工作线程
QObject	ckernel, AudioRead, AudioWrite, VideoRead, ScreenRead, H264Encoder, H264Decoder, INetMediator, ThreadWorker	Qt 对象基类
QDialog	LoginDialog, WeChatDialog, RoomDialog	Qt 对话框基类
QWidget	UserShow	Qt 窗口部件基类
QOpenGLWidget	YuvGLWidget, MyOpenGLWidget	Qt OpenGL 控件基类

8.2 聚合关系（Aggregation - 弱拥有）

聚合方	被聚合方	多重性	说明
ckernel	INetMediator (m_pClient)	1	信令 TCP 连接
ckernel	INetMediator (m_pAVClient)	2	音频+视频 TCP 连接
ckernel	LoginDialog	1	登录对话框
ckernel	WeChatDialog	1	主界面对话框
ckernel	RoomDialog	1	房间对话框
ckernel	AudioRead	1	音频采集器
ckernel	AudioWrite	0..*	音频播放器（每成员一个）

聚合方	被聚合方	多重性	说明
ckernel	VideoRead	1	摄像头采集器
ckernel	ScreenRead	1	桌面共享采集器
ckernel	H264Encoder	1	H264 编码器
ckernel	H264Decoder	0..*	H264 解码器（每成员一个）
ckernel	SendVideoWorker	1	视频发送工作线程
INetMediator	INet	1	网络底层实现
RoomDialog	UserShow	1..*	用户显示控件（每成员一个）

8.3 组合关系（Composition - 强拥有）

组合方	被组合方	多重性	说明
UserShow	YuvGLWidget	0..1	YUV 渲染控件（延迟创建）
AVSyncManager	SyncContext	0..*	同步上下文（每用户一个）
ckernel	H264FragmentGroup	0..*	分片重组缓冲区（内部结构体）

8.4 依赖关系（Dependency - 使用）

依赖方	被依赖方	说明
ckernel	AVSyncManager	单例访问同步管理器
ckernel	MD5	密码加盐哈希
TcpClientMediator	TcpClient	创建 TcpClient 实例
TcpServerMediator	TcpServer	创建 TcpServer 实例
UdpMediator	UdpNet	创建 UdpNet 实例
VideoRead	MyFaceDetect	可选人脸检测
ckernel	H264Encoder	编码调度
ckernel	H264Decoder	解码管理

8.5 关联关系（Association - 结构关系）

关联方	被关联方	说明
INet	INetMediator	m_pMediator 反向关联中介者

九、关键设计说明

9.1 三路 TCP 连接架构

```
ckernel
  ├── m_pClient（信令连接）──→ TcpClientMediator ──→ TcpClient ──→ 服务器:8000
  │     用途：登录/注册/房间管理/成员通知
  │     特点：低频小包，要求即时响应
  │
  ├── m_pAVClient[0]（音频连接）──→ TcpClientMediator ──→ TcpClient ──→ 服务器:8000
  │     用途：Opus 音频帧转发
  │     特点：高频小包（50帧/秒），要求低延迟
  │
  └── m_pAVClient[1]（视频连接）──→ TcpClientMediator ──→ TcpClient ──→ 服务器:8000
        用途：H264 视频帧分片转发
        特点：中频大包（30帧/秒，4KB/分片），要求高吞吐
```

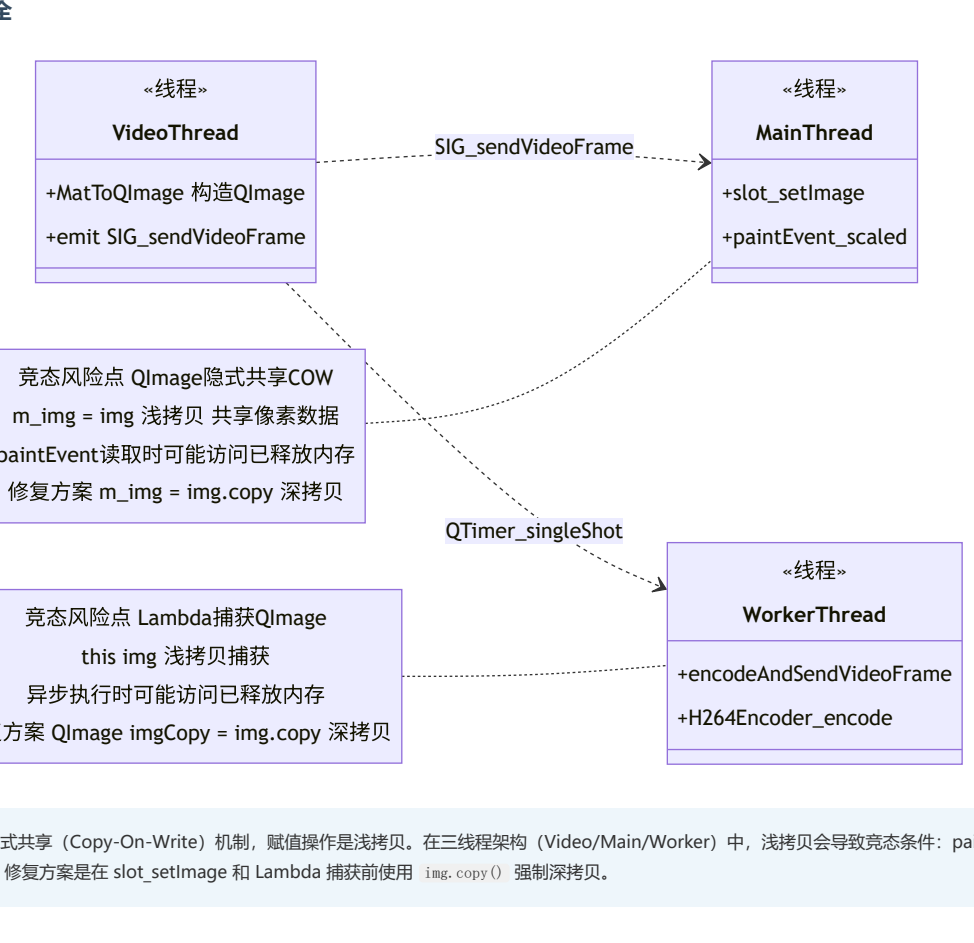
> **设计说明：** 三路 TCP 连接分离信令与媒体流，避免大视频帧阻塞信令通道。每路连接独立的 TcpClientMediator 实例，通过 SIG_ReadyData 信号统一汇入 ckernel::slot_dealData 处理。

9.2 双路径视频渲染架构

```
ckernel::slot_sendVideoFrame(QImage)
  │
  ├── H264 路径（m_useH264 = true）
  │     ├── H264Encoder::encode(QImage) → QByteArray
  │     ├── 按 4KB 分片 → STRU_VIDEO_FRAME_H264_HEADER + h264Data
  │     ├── SendVideoWorker 子线程发送
  │     └── 接收端：H264Decoder::decode → AVFrame → YuvGLWidget GPU 渲染
  │
  └── JPEG 路径（m_useH264 = false，兼容回退）
        ├── QImage::save("JPG") → QByteArray
        ├── STRU_VIDEO_FRAME + jpegData
        └── 接收端：slot_refreshUser(QImage) → QPainter CPU 渲染
```

> **设计说明：** H264 路径使用 GPU 加速渲染（YuvGLWidget + OpenGL Shader），JPEG 路径使用 CPU 软渲染（QPainter）。UserShow 控件通过 m_glWidget 的可见性自动切换渲染路径。

9.3 QImage 隐式共享线程安全


«线程»
VideoThread
+MatToQImage 构造QImage
+emit SIG_sendVideoFrame
SIG_sendVideoFrame
«线程»
MainThread
+slot_setImage
+paintEvent_scaled
竞态风险点 QImage隐式共享COW
m_img = img 浅拷贝 共享像素数据
paintEvent读取时可能访问已释放内存
修复方案 m_img = img.copy 深拷贝
QTimer_singleShot
«线程»
WorkerThread
+encodeAndSendVideoFrame
+H264Encoder_encode
竞态风险点 Lambda捕获QImage
this img 浅拷贝捕获
异步执行时可能访问已释放内存
修复方案 QImage imgCopy = img.copy 深拷贝

> **设计说明：** Qt 的 QImage 采用隐式共享（Copy-On-Write）机制，赋值操作是浅拷贝。在三线程架构（Video/Main/Worker）中，浅拷贝会导致竞态条件：paintEvent 读取像素数据时，原始 QImage 可能已被销毁。修复方案是在 slot_setImage 和 Lambda 捕获前使用 `img.copy()` 强制深拷贝。

9.4 FFmpeg 帧缓冲区管理

```
H264Encoder::encode(QImage) 流程：
  │
  ├── 1. QImage 深拷贝：img.copy() → 独立像素数据
  ├── 2. av_frame_make_writable(m_frame) → 确保 YUV 帧缓冲区可写
  ├── 3. sws_scale(m_swsCtx, RGB→YUV420P) → 色彩空间转换
  ├── 4. avcodec_send_frame(m_codecCtx, m_frame) → 发送帧到编码器
  └── 5. avcodec_receive_packet(m_codecCtx, m_pkt) → 接收编码后的包

H264Decoder::decode(QByteArray) 流程：
  │
  ├── 1. avcodec_send_packet(m_codecCtx, m_pkt) → 发送包到解码器
  ├── 2. avcodec_receive_frame(m_codecCtx, m_frame) → 接收解码后的帧
  └── 3. av_frame_clone(m_frame) → 引用计数 +1，零拷贝跨线程传递
```

> **设计说明：** 编码器使用 `av_frame_get_buffer` + `av_frame_make_writable` 标准组合管理帧缓冲区，支持引用计数。解码器使用 `av_frame_clone` 实现零拷贝跨线程传递，避免像素数据复制开销。

> **文档版本：** v2.0 **最后更新：** 2026-07-29 **数据来源：** 源码头文件静态分析（ckernel.h, INet.h, TcpClient.h, INetMediator.h, audioread.h, audiowrite.h, videoread.h, screenread.h, h264encoder.h, h264decoder.h, yuvglwidget.h, myopenglwidget.h, AVSyncManager.h, threadworker.h, md5.h, roomdialog.h, usershow.h, logindialog.h, wechatdialog.h, packdef.h） **重构内容：** 基于源码头文件全面重构类图，完整标注每个类的属性（名称、类型、可见性）和方法（名称、参数、返回值、可见性），使用标准 UML 符号标识所有类间关系（继承、实现、关联、聚合、组合、依赖），添加设计模式注释和关键设计说明。